[Query] Convert Row <-> Array
unnest (see SQL 2023 7.6 <table reference>) function for array -> row
subquery for row -> array

```
-- Rows to array
SELECT ARRAY (VALUES (1), (2));

 array
═══════
 {1,2}


-- with array_agg
SELECT array_agg(x) x
FROM (VALUES (1), (2));

   x
═══════
 {1,2}

SELECT x, array_agg(y) y
FROM (VALUES (1,'a'), (1,'b')) t(x,y)
GROUP BY x;

 x |   y
───┼───────
 1 | {a,b}



-- Array to rows
SELECT x
FROM UNNEST (ARRAY[1,2]) t(x);

 x
═══
 1
 2
```

[Query] select rows by row number
Use **ROW_NUMBER()** window function

```
-- e.g. every 3rd row only
SELECT (t).mytable.*                            -- unwrap
table row
FROM (
  SELECT
    ROW_NUMBER() OVER (ORDER BY n) as r,        -- row
number
    mytable FROM mytable                        -- table
row
) t
WHERE t.r % 3 = 0
```

[Query] order of elements in ARRAY_AGG()
https://stackoverflow.com/questions/36347637/array-agg-guaranteed-consistent-across-multiple-columns-in-postgres

By default, **ARRAY_AGG()** is arbitrary order

```
-- Default: arbitrary order
SELECT
   parent.id,
   ARRAY_AGG(child.id),                                  --
arbitrary order
   ARRAY_AGG(child.name)
FROM parent
INNER JOIN child ON parent.id = child.parent_id
GROUP BY parent.id


-- Solution 1: ORDER BY in ARRAY_AGG
SELECT
   parent.id,
   ARRAY_AGG(child.id ORDER BY child.id),          --
ORDER BY
   ARRAY_AGG(child.name ORDER BY child.id)
FROM parent
INNER JOIN child ON parent.id = child.parent_id
GROUP BY parent.id


-- Solution 2: ORDER BY in subquery
SELECT
   parent.id,
   ARRAY_AGG(child.id),
   ARRAY_AGG(child.name)
FROM parent
INNER JOIN (
   SELECT child.id, child.name, child.parent_id
   FROM child
   ORDER BY child.id                                     --
ORDER BY
) child ON  parent.id = child.parent_id
GROUP BY parent.id
```

Simulate assertion
Ref: https://hakibenita.com/future-proof-sql#failing-without-a-function

Use casting errors

```
CASE x
  WHEN ... THEN ...
  ELSE CAST(x AS INT) -- cast to an impossible type
END
```

[Table] Update with reference to another table
Ref: https://stackoverflow.com/questions/48434092/sql-update-case-without-a-join
Use **MERGE** or **UPDATE ... FROM**

```
-- BAD: may set NULL
UPDATE t1
  SET value=(                  -- value=NULL if no match
    SELECT value
    FROM t2
    WHERE t2.id = t1.id
 )


-- OK, but not great: with WHERE query too
UPDATE t1
  SET value=(                  -- value=NULL if no match
    SELECT value
    FROM t2
    WHERE t2.id = t1.id
 )
 WHERE EXISTS (
    SELECT 1 WHERE t2.id = t1.id
 )

-- GOOD: with MERGE
MERGE INTO t1
USING t2
ON t1.id = t2.id
WHEN MATCHED THEN
 UPDATE SET value = t2.value


-- GOOD [Non-standard]: JOIN
UPDATE t1
  SET value=t2.value
  FROM t2                          -- LEFT JOIN
  WHERE t1.id = t2.id
```

[Table] create a copy of some rows by changing some columns
Ref: https://stackoverflow.com/questions/15972743/inserting-from-a-select-but-changing-one-column
Create a **TEMPORARY** table with rows to copy

```
-- 1) get rows to copy
CREATE TEMPORARY TABLE temp AS (SELECT * FROM mytable
WHERE ...);

-- 2) change some columns
UPDATE temp SET x=2, y='a';
ALTER TABLE temp DROP id;              -- e.g. delete
autogenerated
```

```
INSERT INTO mytable SELECT * FROM temp;
```

[Table] Delete with reference to another table
Use subquery: https://stackoverflow.com/questions/11753904/postgresql-delete-with-inner-join

```
DELETE FROM mytable1
WHERE id IN (

   -- list of ids with no match in mytable2
   SELECT todelete.id FROM (
     SELECT m1.id as id, m2.id as id2 FROM mytable1 m1
LEFT JOIN mytable2 m2 ON m1.id = m2.id
   ) AS todelete
  WHERE todelete.id2 IS NULL
)
```

==========
======